Contents

Go Best Practices Guide

This document explains the intermediate Go concepts and coding conventions used throughout this codebase. It is designed for developers who know basic Go but want to understand the idioms and patterns used here.

> **Note:** This codebase follows Go best practices and idiomatic patterns. For explanations of intermediate Go concepts used in the code, see GO_CONCEPTS.md. This guide provides deeper explanations and context.

Table of Contents

Error Handling

The Go Error Pattern

In Go, errors are values, not exceptions. Functions return errors as the last return value:

```go
// Standard Go pattern: (result, error) or (result1, result2, error)
func GetMetrics() (*Metrics, error) {
    // ... code ...
    if err != nil {
        return nil, err  // Return nil result and the error
    }
    return metrics, nil  // Return result and nil error
}
```

Why this pattern? - Forces explicit error handling (no hidden exceptions) - Makes error handling visible in the code - Allows multiple return values (result + error)

Checking Errors

Always check errors immediately:

```go
//  GOOD: Check error immediately
metrics, err := GetMetrics()
if err != nil {
    return nil, err  // Handle or propagate
}
// Use metrics here

//  BAD: Ignoring errors
metrics, _ := GetMetrics()  // Do not do this!
```

Best Practice: Never ignore errors. If you must, document why with a comment.

In this codebase: When errors are intentionally ignored, a comment explains why. Examples: cache save in `internal/api/client.go` ("Save to cache (ignore errors - caching is best effort)") and timezone in `internal/cli/cache_commands.go` ("Timezone is best-effort for display only… We ignore LoadTimezone errors so inspect works even with invalid/missing timezone in config").

Pointers and Value Semantics

When to Use Pointers

Use pointers (`*Type`) when: 1. The value is large (structs with many fields) 2. You need to modify the value 3. The value can be `nil` (optional) 4. You want to avoid copying (performance)

Use values when: 1. The value is small (primitives, small structs) 2. You do not need to modify it 3. Immutability is desired

Examples in This Codebase

```go
// Pointer receiver - modifies the struct, avoids copying
func (c *EnlightenCloudClient) GetMetrics() (*LocalMetrics, error) {
    // 'c' is a pointer - changes to c affect the original
}

// Value return - small struct, no modification needed
func getDefaultColors() ColorConfig {
    return ColorConfig{...}  // Return by value
}

// Pointer return - large struct, may be nil
func LoadConfig() (*Config, error) {
    config := &Config{}  // Create pointer
    return config, nil   // Return pointer
}
```

Struct Methods and Receivers

Receiver Types

Go methods are functions with a special "receiver" parameter:

```go
// Value receiver - receives a copy
func (d Display) ShowInfo(msg string) {
    // Changes to 'd' do not affect the original
}

// Pointer receiver - receives a reference
func (c *EnlightenCloudClient) GetMetrics() (*LocalMetrics, error) {
    // Changes to 'c' affect the original
    c.cacheUsed = true  // Modifies the original struct
}
```

When to Use Each

Use pointer receivers (`*Type`) when: - Method modifies the struct - Struct is large (avoids copying) - Consistency (if any method uses pointer, all should)

Use value receivers (`Type`) when: - Method does not modify the struct - Struct is small - You want immutability

In this codebase: We use pointer receivers for all struct methods because: 1. Methods often modify state (e.g., `cacheUsed` flag) 2. Structs are moderately sized 3. Consistency across the codebase

---

Channels and Goroutines

> **For a comprehensive deep dive into channels, signals, and the `select` statement as used in this codebase, see GO_CONCEPTS.md**. This section provides a brief overview of the concepts.

Channels

Channels are Go's way to communicate between goroutines safely. In this codebase, we use `signal.NotifyContext` which creates a context that is cancelled when signals are received.

Channel Types: - `chan T` - bidirectional (can send and receive) - `chan<- T` - send-only - `<-chan T` - receive-only

Select Statement

`select` lets you wait on multiple channel operations:

```
select {
case <-ticker.C:
    // Timer ticked - do periodic work
case <-ctx.Done():
    // Signal received - handle shutdown
    return
}
```

Why use select? - Non-blocking: waits for first available case - Allows handling multiple channels - Essential for graceful shutdown

> **For detailed examples, real-world scenarios, and edge cases, see GO_CONCEPTS.md**.

Goroutine lifetimes

When you spawn a goroutine, make it clear **when or whether it exits**. Every goroutine should have a predictable stop mechanism and a way to wait for it to finish.

- **Don't fire-and-forget** — Use a done channel or `sync.WaitGroup` so callers can wait for the goroutine to exit.
- **Prefer synchronous code** — Keep concurrency at the caller: use synchronous functions (e.g. a `for`/`select` loop) and let the caller add goroutines if needed. In this codebase, `RunContinuous` is a synchronous loop that uses `select` on `ticker.C` and `ctx.Done()`; it does not spawn goroutines.
- **No goroutines in init()** — Don't start background goroutines in `init()`. Use an object with an explicit `Close`/`Stop`/`Shutdown` method if you need a long-lived worker.

In this codebase: The only spawned goroutine is in tests (e.g. a goroutine that cancels the context after a delay). That goroutine uses a `done` channel and the test waits on `<-done` before ending. Production code uses a synchronous `select` loop in `RunContinuous` for periodic refresh and shutdown.

JSON Struct Tags

The Problem

Go uses PascalCase for exported fields, but JSON APIs often use snake_case:

```
type SystemConfig struct {
    Name string // Go convention: PascalCase
    ID   string
}
// JSON API returns: {"name": "My System", "id": "12345"}
```

The Solution: Struct Tags

Struct tags tell the JSON encoder/decoder how to map fields:

```
type SystemConfig struct {
    Name string `json:"name"` // Maps to "name" in JSON
    ID   string `json:"id"`   // Maps to "id" in JSON
}
```

Tag Syntax: - Backticks: `` `json:"field_name"` `` - Multiple tags: `` `json:"field_name" xml:"FieldName"` `` - Omit if empty: `` `json:"field_name,omitempty"` ``

Common Tags in This Codebase

```
type TelemetryResponse struct {
    LastReportedAggregateSOC string              `json:"last_reported_aggregate_soc,omitempty"
    Intervals                []TelemetryInterval `json:"intervals"`
}
```

Defer Statements

What is Defer?

`defer` schedules a function call to execute when the surrounding function returns:

```
func ReadFile(filename string) ([]byte, error) {
    file, err := os.Open(filename)
    if err != nil {
        return nil, err
    }
    defer file.Close()  // Always closes, even if function returns early

    // ... read file ...
    return data, nil  // file.Close() executes here
}
```

Defer Execution Order

Defer calls execute in LIFO (Last In, First Out) order:

```go
defer fmt.Println("First")
defer fmt.Println("Second")
defer fmt.Println("Third")
// Output when function returns:
// Third
// Second
// First
```

Common Uses

1. **Resource cleanup:**

   ```go
   resp, err := http.Get(url)
   if err != nil {
       return err
   }
   defer resp.Body.Close()  // Always close response body
   ```

2. **Stopping timers:**

   ```go
   ticker := time.NewTicker(interval)
   defer ticker.Stop()  // Always stop ticker
   ```

Slice Capacity Hints

The Problem

When you know how many elements a slice will hold, pre-allocating capacity is more efficient:

```go
//  SLOW: Slice grows multiple times
systems := []SystemMetrics{}
for _, sys := range config.Systems {
    systems = append(systems, metrics)  // May reallocate multiple times
}

//  FAST: Pre-allocate capacity
systems := make([]SystemMetrics, 0, len(config.Systems))
//                               ^length ^capacity
for _, sys := range config.Systems {
    systems = append(systems, metrics)  // No reallocation needed
}
```

How It Works

```go
// make([]Type, length, capacity)
systems := make([]SystemMetrics, 0, 10)
//                                ^    ^
//                                |      Can hold 10 elements without reallocating
//                                  Currently has 0 elements
```

In this codebase:

```
// aggregator.go - we know exactly how many systems we have
Systems: make([]SystemMetrics, 0, len(systems)),
//                              ^    ^
//                              |      Capacity = number of systems
//                                Start with empty slice
```

Map capacity

When building a map from a known-size collection, use a capacity hint to reduce reallocations:

```
// cache_commands.go - unique dates from entries
dateSet := make(map[string]bool, len(allEntries))
dates := make([]string, 0, len(dateSet))
```

strconv over fmt for primitives

For converting integers to strings (e.g. URL query params, ANSI codes), `strconv` is faster than `fmt.Sprintf`:

```
// client.go, urlbuilder.go - telemetry URL query params
"&start_at=" + strconv.FormatInt(dayStart.Unix(), 10) + "&end_at=" + strconv.FormatInt(dayEnd.U
// config.go - ANSI escape code
"\033[38;5;" + strconv.Itoa(ansiCode) + "m"
```

Interface Satisfaction

Implicit Interfaces

Go interfaces are satisfied **implicitly** - no `implements` keyword needed:

```
// io.Reader interface (from standard library)
type Reader interface {
    Read(p []byte) (n int, err error)
}

// Any type with a Read method automatically satisfies io.Reader
type MyReader struct{}

func (r MyReader) Read(p []byte) (n int, err error) {
    // ... implementation ...
}
// MyReader now satisfies io.Reader - no explicit declaration needed!
```

Why This Matters

```
// http.Response.Body is an io.Reader
// We can pass it to any function expecting io.Reader
func ReadAll(r io.Reader) ([]byte, error) {
```

```
    // Works with http.Response.Body, os.File, bytes.Buffer, etc.
}

resp, _ := http.Get(url)
data, _ := ReadAll(resp.Body)  // resp.Body satisfies io.Reader
```

Benefits: - Loose coupling: functions depend on behavior, not types - Easy testing: can create mock implementations - Flexible: types can satisfy multiple interfaces

---

Package-Level Variables

What Are They?

Variables declared outside functions at package level:

```
package main

// Package-level variables
var (
    tokenCache *TokenCache  // Shared cache
)
```

When to Use

Use package-level variables for: - Shared state across functions (caches, configuration) - Configuration that does not change - Constants

Avoid for: - Data that should be passed as parameters - State that should be encapsulated in structs

In This Codebase

```
// oauth.go - token cache shared across all OAuth operations
var (
    tokenCache *TokenCache  // Shared cache (accessed from main goroutine only)
)

// internal/cache/cache.go - configuration flags
var (
    testMode      bool  // When true, only use cached responses (no live API calls)
    cacheDisabled bool  // When true, always make live API calls
)
```

Why package-level here? - These are singletons (one cache, one set of flags) - Used across multiple functions - Need to persist between function calls - In this codebase, all shared state is accessed from the main goroutine only (single-threaded execution)

Error Wrapping

The Problem

When propagating errors, you want to add context without losing the original error:

```
//  BAD: Loses original error
if err != nil {
    return fmt.Errorf("failed to get metrics")  // Original error lost!
}

//  GOOD: Wraps original error
if err != nil {
    return fmt.Errorf("failed to get metrics: %w", err)  // Preserves original
}
```

The %w Verb

The `%w` verb in `fmt.Errorf` wraps errors, preserving the error chain:

```
func GetMetrics() (*Metrics, error) {
    data, err := fetchData()
    if err != nil {
        // Wrap with context, preserve original error
        return nil, fmt.Errorf("failed to fetch data: %w", err)
    }
    return parseData(data)
}

// Caller can inspect the error chain:
metrics, err := GetMetrics()
if err != nil {
    // Can check for specific error types
    if errors.Is(err, io.EOF) {
        // Handle EOF specifically
    }
    // Or unwrap to get original error
    originalErr := errors.Unwrap(err)
}
```

Error Chain Example

```
// internal/cache/cache.go
if err != nil {
    return nil, fmt.Errorf("failed to read cache file: %w", err)
}

// internal/api/client.go
if err != nil {
    return nil, fmt.Errorf("failed to get metrics: %w", err)
```

```
}

// aggregator.go
if err != nil {
    return nil, fmt.Errorf("failed to get metrics from Cloud API: %w", err)
}

// Error chain when it reaches main.go:
// "failed to get metrics from Cloud API: failed to get metrics: failed to read cache file: fi
```

Time Handling

Time Types

Go has several time-related types:

```
time.Time      // Represents a point in time
time.Duration  // Represents a duration (nanoseconds)
*time.Time     // Pointer to time (allows nil = "not set")
```

Common Patterns

1. Creating time values:

```
now := time.Now()                    // Current time
midnight := time.Date(2026, 1, 15, 0, 0, 0, 0, time.UTC)
duration := 5 * time.Minute          // Duration literal
```

2. Parsing dates:

```
// Go's reference time: Mon Jan 2 15:04:05 MST 2006
//                     = 01/02 03:04:05PM '06 -0700
date, err := time.Parse("2006-01-02", "2026-01-15")
```

3. Formatting dates:

```
t := time.Now()
fmt.Println(t.Format("2006-01-02"))           // "2026-01-15"
fmt.Println(t.Format("Mon Jan 2, 2006"))      // "Wed Jan 15, 2026"
fmt.Println(t.Format("03:04:05 PM"))          // "02:30:45 PM"
```

4. Timezone handling:

```
utc := time.Now().UTC()
reportTZ, _ := LoadTimezone(config.Timezone) // From config, system, or US/Pacific fallback
local := time.Now().In(reportTZ)
```

Pointer to Time (Optional Time)

```
// *time.Time allows nil = "not set"
type AggregatedMetrics struct {
    Timestamp time.Time   // Always has a value
```

```
    QueryDate *time.Time  // Can be nil (means "today")
}

// Usage:
var queryDate *time.Time  // nil = use today
if userSpecifiedDate {
    d := time.Date(2026, 1, 15, 0, 0, 0, 0, time.UTC)
    queryDate = &d  // Set to specific date
}
```

Go Packages and Imports

This section summarizes package organization and import conventions (go-packages skill).

Exit only from main

Call `os.Exit` or `log.Fatal` **only in `main()`**. All other code should return errors so callers (and tests) can handle them. Prefer a single exit point in main (e.g. `if err := run(); err != nil { log.Fatal(err) }`).

In this codebase:

- **main.go** is the only file that calls `os.Exit(1)`. It handles errors from config, OAuth, cache commands, `RunOnce`, and `RunContinuous` by printing to stderr and exiting.
- **internal/app/runner.go** – `RunOnce` and `RunContinuous` return `error` instead of exiting. `fetchAndDisplay` returns `error` so the continuous loop can stop on fatal errors (e.g. rate limit).
- **internal/app/setup.go** – No `ExitWithError`; main performs the fprintf and exit.

Benefits: Testable code (no exit in tests), predictable control flow, `defer` in helpers always runs.

Import organization

- **Standard library first**, then a blank line, then project imports. Use **goimports** to maintain order.
- **Rename imports** only when necessary (e.g. name collision); prefer renaming the more local or project-specific import.
- **Blank imports** (`import _ "pkg"`) only in main or tests that need side effects (e.g. drivers).
- **No dot imports** (`import . "pkg"`) except in tests that must live in `package foo_test` and avoid circular dependencies.

Avoid init()

Prefer explicit setup (e.g. `loadConfig()` called from main) over `init()`. If you use `init()`, keep it deterministic and free of I/O, env, or global state.

Package names

Use descriptive package names (e.g. `urlbuilder`, `timezone`, `display`). Avoid generic names like `util`, `helper`, or `common`.

Go Naming Conventions

This section summarizes naming rules for packages, types, functions, receivers, constants, and variables (go-naming skill).

MixedCaps, no underscores

All Go identifiers use **MixedCaps** or **mixedCaps** (camelCase). No underscores (no snake_case) in identifier names.

Exceptions: Test and benchmark names may use underscores: `TestFoo_InvalidInput`, `BenchmarkSort_LargeSlice`. Filenames are not identifiers and may use underscores (e.g. `client_functional_test.go`).

Package names

- **Lowercase only**, no underscores: `urlbuilder`, `timezone`, `display`.
- **Descriptive**, not generic: prefer `stringutil`, `httpauth` over `util`, `common`, `helper`.

Receiver names

- **Short (1–2 letter) abbreviations** of the type, **consistent** for that type.
- Examples in this codebase: `(c *EnlightenCloudClient)`, `(a *DataAggregator)`, `(d *Display)`, `(m *MockCloudClient)`.

Constants

- **MixedCaps**, never `ALL_CAPS` or a `k` prefix: `MaxRetries`, `defaultTimeout`, not `MAX_RETRIES` or `kMaxRetries`.
- Name by **role**, not value: `MaxRetries`, `DefaultPort` rather than `Three`, `Port8080`.

In this codebase: `constants` package uses MixedCaps throughout (`Reset`, `Bold`, `DateFormat`, `APIRequestTimeout`, `HTTPStatusOK`).

Initialisms and acronyms

- Keep case **consistent** in the name: all caps or all lowercase for the acronym.
- Exported: `URL`, `ID`, `API`, `HTTP`. Unexported: `url`, `id`, `api`, `http`.
- Examples: `HTTPClient`, `userID`, `ParseURL()`; avoid `HttpClient`, `orderId`, `ParseUrl()`.

Getters and actions

- **Simple accessors:** use `Owner()` not `GetOwner()`. **Setters:** `SetOwner()`.
- **Non-trivial or I/O:** `Get` or `Fetch`/`Compute` is appropriate: `GetMetricsFromCloud()`, `GetEnergyImportForDate()` (they perform I/O), so the `Get` prefix is correct here.

Avoiding repetition

- **Package + symbol:** `urlbuilder.BuildTelemetryURL()` not `urlbuilder.BuildTelemetryURLForSystem(` when context is clear.
- **Receiver + method:** `c.buildTelemetryURL()` not `c.buildClientTelemetryURL()` when the receiver is the client.

Go Interfaces and Composition

This section summarizes interface usage, type assertions, embedding, and receiver consistency (go-interfaces skill).

Implicit interface satisfaction

Go has no `implements` keyword. A type satisfies an interface by implementing its methods. **In this codebase:** `*EnlightenCloudClient` implements `api.CloudClient` by defining all interface methods (e.g. `GetMetricsFromCloud`, `GetEnergyImportForDate`). Mocks (e.g. `MockCloudClient` in tests) implement the same interface for dependency injection.

Interface naming

One-method interfaces use the method name plus **`-er`**: `Reader`, `Writer`, `Stringer`. Multi-method interfaces (e.g. `CloudClient`) use a descriptive name for the capability.

Compile-time interface check

To ensure a type implements an interface at compile time, use a blank identifier assignment:

```
// api/interface.go - fails to build if *EnlightenCloudClient no longer implements CloudClient
var _ CloudClient = (*EnlightenCloudClient)(nil)
```

Use this when there is no other static conversion that would catch the error. **In this codebase:** this check lives in `internal/api/interface.go` next to the `CloudClient` interface.

Type assertions and type switches

- **Type assertion:** `v := x.(Type)` — panics if wrong type. **Safe form:** `v, ok := x.(Type)` — returns zero value and `false` on failure.
- **Type switch:** `switch v := x.(type) { case string: ... case int: ... }` — variable has the correct type in each case.

This codebase does not use type assertions or type switches; it relies on interfaces and dependency injection.

Receiver consistency

Use **pointer receivers** for all methods on a type when any method mutates the receiver or the type is large. **Don't mix** value and pointer receivers on the same type. **In this codebase:** `*EnlightenCloudClient`, `*Display`, `*DataAggregator`, `*ColorConfig`, and test mocks use pointer receivers consistently.

Struct and interface embedding

- **Interface embedding:** combine interfaces: `type ReadWriter interface { Reader; Writer }`.
- **Struct embedding:** embed a type (no field name) to promote its methods: `type S struct { *T }`. The embedded type's name is the field name for access.

This codebase does not use struct or interface embedding; types use named fields and explicit dependency injection (e.g. `DataAggregator` holds `getAccessToken` and `createCloudClient` as named fields).

Functional Options

Functional options is a pattern for constructors and public APIs with **3+ optional arguments**: an unexported `options` struct, an exported `Option` interface with an unexported `apply(*options)` method, and `With*` constructors that return options. The constructor takes required parameters plus `...Option` and applies them over defaults. (See go-functional-options skill; Uber Go Style Guide.)

When to use

- **3+ optional arguments** on constructors or public APIs
- **Extensible APIs** that may gain new options over time
- **Clean caller experience** — callers only pass what differs from defaults

Prefer a config struct or two constructors when: fewer than 3 options, options rarely change, or the API is internal-only.

Pattern (summary)

1. **Unexported `options` struct** — holds all configuration; set defaults before applying options.
2. **Exported `Option` interface** — `apply(*options)` method is unexported so only this package can create valid options.
3. **Option types** — each implements `Option` (e.g. `type baseURLOption string` with `apply(opts *options)`).
4. **`With*` constructors** — e.g. `WithBaseURL(url string) Option`, `WithHTTPClient(c *http.Client) Option`.
5. **Constructor** — `func New(required1, required2 string, opts ...Option) (*Thing, error)`; start with defaults, then `for _, o := range opts { o.apply(&opts) }`.

In this codebase

- **No functional options in use.** Optional configuration is handled with **two constructors** where there is only one or two "options":
  - **api:** `NewEnlightenCloudClient(systemID, apiKey, accessToken, tz)` vs `NewEnlightenCloudClientWithBaseURL(baseURL, systemID, apiKey, accessToken, tz)` — the only "option" is base URL (for tests). Two constructors are clear and acceptable here.

  - **display:** `NewDisplayWithColorsAndTimezone(colors, tz)` (default writer) vs `NewDisplayWithWriter(colors, tz, w)` — one "option" (writer). Acceptable.
- **If the API client gains more options** (e.g. custom HTTP client, timeout, retries), consider refactoring to one constructor plus functional options: `NewEnlightenCloudClient(systemID, apiKey, accessToken, tz, WithBaseURL(url), WithHTTPClient(client))` for extensibility and a single entry point.

Go Error Handling Conventions

This section summarizes error handling rules from the go-error-handling skill (Google/Uber style guides). See also Error Handling and Error Wrapping above.

Return the error type

Exported functions should return the **`error` interface**, not concrete error types (e.g. `*os.PathError`). Returning concrete types can make a `nil` pointer become a non-nil interface and confuse callers.

Error strings

- **Lowercase, no trailing punctuation** — e.g. `"something failed"` not `"Something failed."`
- **Exception:** May start with a capital for exported names, proper nouns, or acronyms (e.g. `"API configuration is required"`).
- **Displayed messages** (logs, test output, API responses) may use normal capitalization.

Handle errors once

Choose **one** response per error:

1. **Return** the error (wrapped with `%w` when adding context) so the caller handles it
2. **Log and degrade** (don't return) when the failure should not propagate
3. **Match** specific errors (e.g. `errors.Is(err, ErrNotFound)`) and handle them; return others

Don't log and return the same error — that causes duplicate logging up the stack.

Error wrapping

- **`%w`** — Use when adding context and you want to preserve the chain for `errors.Is` / `errors.As`. Place **at the end**: `"context message: %w"`.
- **`%v`** — Use at system boundaries or when you want to hide internal detail.

Indent error flow

Handle errors first, then keep the normal path at base indent. Avoid `if err != nil { ... } else { normal }`; use early return so the happy path is unindented.

In this codebase

- Errors are returned as `error`; wrapping uses `%w` at the end of format strings.

- Error strings are lowercase (except where acronyms/exported names lead); the one violation (`"invalid date format. Use"`) was fixed to `"invalid date format: use"`.
- No "log and return" pattern; main prints and exits, other code returns errors.
- Error flow uses early return; no `else`-after-error pattern.

Go Documentation Conventions

This section summarizes documentation rules from the go-documentation skill (Google Go Style Guide). See also Comments and documentation in Coding Conventions.

Package comments

- **One per package** — Every package has exactly one package comment, above the `package` clause (usually in the main file or a single file that godoc picks first).
- **Start with "Package <name>"** — First line should describe what the package does, e.g. `Package cli provides command-line flag parsing and cache management...`.
- **Main package** — Use the binary name; e.g. "Package main implements the Enphase Monitor application" or "The enphase-monitor command…".

Doc comments for exported names

- **Begin with the name** — First sentence starts with the name of the type, function, or constant (e.g. "LoadConfig reads and parses…").
- **Complete sentences** — Capitalize the first word and end with a period.
- **Parameters / cleanup / errors** — Document non-obvious behavior, cleanup requirements, and significant error sentinels or types; don't restate the obvious or implied context behavior.

In this codebase

- Every package has a single package comment; multi-file packages use one canonical comment (e.g. cli in cache_commands.go, aggregator in aggregator.go).
- Exported types and functions have doc comments that start with the name and use full sentences with punctuation.
- Detailed walkthroughs and onboarding style are in GO_BEST_PRACTICES, GO_CONCEPTS, ARCHITECTURE, and TESTING.md; see docs/go-documentation-review.md for the full review.

Go Defensive Programming

This section summarizes defensive programming patterns from the go-defensive skill (Uber/Google style guides). It helps keep the codebase robust and prevents accidental bugs at API boundaries.

Interface compliance

- **Compile-time checks** — Use `var _ Interface = (*Type)(nil)` so that if a type stops implementing an interface, the build fails. This codebase has this for `api.CloudClient` in `internal/api/interface.go`.

Copy slices and maps at boundaries

- **Receiving** — When storing a slice or map from a caller, copy it instead of assigning. Otherwise the caller can mutate your internal state.
- **Returning** — When returning a slice or map that backs internal state, return a copy so callers cannot mutate it. Example: `GetAggregatorTypes` in `internal/app/setup.go` returns a copy of `cfg.Systems` so callers cannot modify the config.

Defer for cleanup

- **Resources** — Use `defer` for `Close()`, `Unlock()`, `Stop()`, etc., so cleanup runs on every return path. This codebase uses `defer resp.Body.Close()` for HTTP responses and `defer ticker.Stop()` for the refresh ticker.

Time and duration

- **Use the time package** — Use `time.Time` for instants and `time.Duration` for intervals. Avoid raw `int` for time values.
- **Unit in name** — When a config field is an integer time value (e.g. seconds), include the unit in the field name so the contract is explicit. Example: `RefreshIntervalSeconds int` with `yaml:"refresh_interval"` (YAML key stays for backward compatibility; the Go name makes the unit clear).

In this codebase

- **Interface check** — `var _ CloudClient = (*EnlightenCloudClient)(nil)` in `internal/api/interface`
- **Slice at boundary** — `GetAggregatorTypes` copies `cfg.Systems` before return.
- **Defer** — HTTP response bodies and the runner ticker use `defer` for cleanup.
- **Time** — Config uses `RefreshIntervalSeconds`; runner uses `time.Duration(...) * time.Second` for the ticker.
- **No math/rand for keys** — No key generation; no crypto/rand vs math/rand concern.
- **Field tags** — Marshaled structs (config, API types, validation) use explicit `json`/`yaml` tags.

Go Data Structures

This section summarizes data-structure conventions from the go-data-structures skill (Effective Go, CodeReviewComments). See also Slice Capacity Hints and Copy slices and maps at boundaries above.

Allocation: new vs make

- **`make`** for slices, maps, and channels — Returns an initialized value (not a pointer). Use `make([]T, length, capacity)` when the size or capacity is known.
- **`new(T)`** returns `*T` zeroed — Use for structs when you need a pointer; avoid `new([]int)` (rarely useful).

Empty slices

- **Prefer nil slice** when declaring an empty slice that will be appended to: `var t []T` rather than `t := []T{}`. They are functionally equivalent (len/cap zero) but the nil slice is the preferred style.
- **Exception:** When encoding to JSON, nil slice → `null`, empty slice → `[]`; use `[]T{}` when you need a JSON array.

append

- **Always assign the result** — `s = append(s, x)`; the underlying array may change, so discarding the return value is a bug.

In this codebase

- **make with capacity** — `aggregator.GetAggregatedMetrics` uses `make([]SystemMetrics, 0, len(systems))`; `cache_commands.showAvailableDates` uses `make(map[string]bool, len(allEntries))` and `make([]string, 0, len(dateSet))`.
- **Nil slices for append** — `var allIntervals []TelemetryInterval` in parser.go and `var rateLimitErrors []string` in aggregator.go, then append; result is always assigned.
- **Copy at boundaries** — `GetAggregatorTypes` returns a copy of the systems slice (see Go Defensive Programming).

Go Context Usage

`context.Context` carries cancellation signals, deadlines, and request-scoped values. Use it for any operation that can be cancelled or that should respect a caller's lifetime.

Context as first parameter

Functions that take a context should accept it as the **first parameter**:

```
// Good: ctx is first
func RunOnce(ctx context.Context, agg *DataAggregator, ...) error { ... }
func GetAccessToken(ctx context.Context, apiConfig *APIConfig) (string, error) { ... }
```

Do not store context in structs

Pass context as an argument to each function that needs it; do not add a `ctx` field to a struct. That keeps context lifetime explicit and avoids confusion when the same struct is reused across different requests.

When to use context.Background()

Use `context.Background()` only for code that is not tied to a request or operation (e.g. `main`, top-level background workers, or one-off setup when no parent context exists). For request-like flows, pass a context from the caller so cancellation (e.g. Ctrl+C) propagates.

```
// Good: main creates signal context for run loop
ctx, stop := signal.NotifyContext(context.Background(), syscall.SIGINT, syscall.SIGTERM)
```

```
defer stop()
app.RunOnce(ctx, agg, disp, ...)

// Good: OAuth setup receives ctx so Ctrl+C cancels token exchange
oauth.Setup(ctx, cfg)
```

In this codebase

- **First parameter** — All context-using functions take `ctx context.Context` as the first parameter (`RunOnce`, `GetAccessToken`, `ExchangeAuthorizationCode`, `GetMetricsFromCloud`, etc.).
- **No context in structs** — `EnlightenCloudClient`, `DataAggregator`, and `Display` do not store context; it is passed into methods.
- **Propagation** — `main` creates a signal context and passes it to `RunOnce`/`RunContinuous` and to `oauth.Setup` so shutdown and Ctrl+C cancel in-flight work.

Coding Conventions Used in This Codebase

This section summarizes the coding conventions and best practices followed throughout this codebase.

1. Error Handling

- **Always check errors immediately** - Do not ignore them
- **Wrap errors with context** - Use `%w` verb in `fmt.Errorf()` to preserve error chain
- **Return errors, do not log and continue** - Let callers decide how to handle errors
- **Fail fast** - Return errors early rather than continuing with invalid state

2. Naming Conventions

- **Exported** (public): PascalCase (`GetMetrics`, `Config`, `SystemMetrics`)
- **Unexported** (private): mixedCaps (`getCacheKey`, `tokenCache`, `parseResponse`)
- **Constants**: MixedCaps only (`Reset`, `Bold`, `DateFormat`, `APIRequestTimeout`) — never `ALL_CAPS` or `k` prefix
- **Interfaces**: Usually end with `-er` (`Reader`, `Writer`, `Closer`)
- **Constructors**: Prefix with `New` (`NewDisplayWithColorsAndTimezone`, `NewEnlightenCloudClient`)
- **Receivers**: Short 1–2 letter abbreviations (`c` for Client, `d` for Display); see Go Naming Conventions

3. Function Organization

- **Small, focused functions** - One responsibility per function
- **Functions return early on errors** - Reduces nesting, improves readability
- **Exported functions have doc comments** - Explain purpose, parameters, return values
- **Complex logic has inline comments** - Explain "why", not just "what"

4. **Control Flow (Reduce Nesting)**

- **Keep nesting to at most 2 levels** - Use early returns and `continue` in loops so the happy path stays unindented
- **Omit else when if returns** - When an `if` body ends with `return`, `break`, or `continue`, write the success path after the `if` instead of in an `else` (guard clauses)
- **If with initialization** - Use `if x := f(); x != nil { }` to scope variables to the conditional; common for errors
- **Prefer default + override over if/else** - When a variable is set in both branches, set a default then override in one branch (e.g. `x := default; if condition { x = other }`)
- **Handle errors and edge cases first** - Return or continue early; keep the main logic at the top level

5. **Comments and documentation**

- **Package comments** - Every package has exactly one package comment describing its purpose (e.g. "Package display provides terminal output formatting…").
- **Doc comments** - Exported names have doc comments that begin with the name of the thing and use full sentences (capitalized, ending with a period).
- **Complex logic** - Inline comments explain the reasoning ("why"), not just the code ("what").
- **Go concepts** - See GO_CONCEPTS.md for explanations of intermediate Go concepts used in the code.

Onboarding: GO_BEST_PRACTICES, GO_CONCEPTS, ARCHITECTURE, and TESTING.md are intentionally detailed (walkthroughs, patterns, examples) to help engineers new to Go get up to speed. When adding docs, keep this style: explain the pattern, show an example, and point to where it appears in the codebase.

6. **Struct Design**

- **Group related fields** - Logical organization
- **Use meaningful field names** - Self-documenting code
- **Add JSON tags for API structs** - Map to API field names
- **Document exported structs** - Explain purpose and usage

7. **Resource Management**

- **Always use `defer` for cleanup** - Files, HTTP bodies, timers
- **Close resources explicitly** - Do not rely on garbage collection
- **Stop timers and tickers** - Prevent resource leaks

8. **Pointers vs Values**

- **Use pointers for:**
  - Large structs (avoid copying)
  - When you need to modify the value
  - Optional values (nil = not set)
  - Consistency (if any method uses pointer receiver, all should)
- **Use values for:**
  - Small primitives (int, string, bool)

  - When you do not need to modify
  - When immutability is desired

9. Error Wrapping

- **Always use `%w` verb** - Preserves error chain for debugging
- **Add context at each level** - "failed to X: %w" pattern
- **Do not wrap unnecessarily** - Only add context that is useful

10. Testing

- **Test files end with `_test.go`** - Go test runner convention
- **Test functions start with `Test`** - `func TestFunctionName(t *testing.T)`
- **Use table-driven tests** - Test multiple cases in one function
- **Validate against expected values** - This codebase uses `internal/validation/validation.go`
- **Test refactor helpers** - Helpers introduced during go-style-core refactoring (e.g. `findSystemByID`, `runMetricTests`, `tryAppendEntryByCachedAt`, `tryLoadPastDateCache`) have dedicated unit tests; see TESTING.md § Testing Refactor Helpers.

Test File Organization This codebase uses two test file organization patterns:

Standard Pattern (1:1 mapping) Most packages follow the simple convention: - `config.go` → `config_test.go` - `constants.go` → `constants_test.go` - `cli.go` → `cli_test.go`

Complex Pattern (1:many mapping) For packages with many functions or complex logic, tests are split by **test category**:

Cache Package (`cache.go`): 1. **`cache_test.go`** - State management tests - Tests state flags (testMode, cacheDisabled, rateLimitWarning) - Tests ResetState() for test isolation

2. **`cache_functions_test.go`** - Core functionality tests
   - Tests URL redaction, cache saving/loading, normalization
   - Tests file operations, error handling
   - Original functional tests

Why split? Cache has two distinct concerns: state management vs core caching functions.

OAuth Package (`oauth.go`): 1. **`oauth_test.go`** - Basic unit tests (270 lines) - Tests GetAuthorizationURL validation - Tests token refresh mechanics - Original tests from early rounds

2. **`oauth_functional_test.go`** - Integration/functional tests (598 lines)
   - Uses mock HTTP servers (`httptest.NewServer`)
   - Tests complete token exchange flows
   - Tests real HTTP interactions with mocked backend
   - Created in Rounds 4-6 for comprehensive coverage
3. **`oauth_edge_cases_test.go`** - Edge case & error path tests (442 lines)
   - Tests validation errors (missing config, empty fields)
   - Tests network errors, timeouts, malformed responses
   - Created in Round 10 Phase A to improve coverage

Why split? OAuth is complex (270 lines of implementation): - Basic validation separate from integration tests - Happy path (functional) separate from error paths (edge cases) - Easier to maintain and understand test intent

Benefits of This Approach:

Clarity - Test file name indicates test category **Maintainability** - Related tests grouped together **Readability** - Smaller files easier to navigate (161-598 lines vs 516-1310 lines) **History** - Shows evolution (original → functional → edge cases) **Focus** - Can run specific test categories independently

Bottom line: The 1:many pattern emerged naturally during refactoring to keep test files organized and maintainable as coverage improved from 29.8% → 70.4%. It's a pragmatic choice, not a strict rule.

Additional Resources

- Effective Go - Official Go best practices
- Go Code Review Comments - Common style guide
- Go by Example - Practical examples